Introduction:

Water wave animation is an important topic in computer graphics used to create realistic moving water effects such as rivers, seas, and oceans. In this project, sinusoidal wave equations are used to generate smooth and continuous water motion. The animation creates the visual effect of flowing water by changing the wave position over time.

The main purpose of this project is to simulate natural water movement using graphical programming techniques. A boat or floating object is also added to make the animation more realistic and visually attractive. The project is implemented using the C++ programming language and the OpenGL graphics library.

This project helps in understanding important concepts of computer graphics such as animation, trigonometric functions, coordinate systems, rendering, and real-time motion simulation. It also demonstrates how mathematical functions can be used to create natural-looking graphical effects in interactive applications.

Problem Statement:

In computer graphics, creating realistic water movement is a challenging task. Static water images do not look natural because real water continuously moves with waves and flow patterns. Without animation, rivers and seas in graphical applications appear unrealistic and less attractive.

This project addresses the problem by using sinusoidal wave equations to generate smooth and continuous water motion. The animation simulates the natural behavior of sea or river waves in real time. Moving waves are created mathematically and updated continuously to produce realistic motion effects.

Additionally, floating objects such as boats are added to increase realism and improve visual appearance. The project demonstrates how mathematical functions and graphics programming techniques can be combined to create dynamic and interactive water animations in computer graphics applications.

Objectives:

The main objectives of this Water Wave Animation project are:

1. To create realistic moving water waves using sinusoidal functions.
2. To simulate sea or river water motion through continuous animation.
3. To understand the use of trigonometric equations in computer graphics.
4. To implement real-time animation using OpenGL and C++.
5. To add floating objects such as boats for improving visual realism.
6. To learn graphical rendering techniques and frame updating methods.
7. To improve knowledge about animation, coordinate systems, and motion effects in computer graphics.
8. To develop an interactive and visually attractive graphics project.

Algorithms Used:

Several algorithms and mathematical techniques are used in this project to create realistic water wave animation.

1. Sinusoidal Wave Algorithm

The main algorithm used in this project is the sinusoidal wave function. It generates smooth and continuous wave patterns by using the sine mathematical function.

The wave equation is:

$$y=A \sin (Bx + Ct)$$

Where:

- **A** = Amplitude (height of the wave)
- **B** = Frequency (number of waves)
- **C** = Speed of wave movement
- **t** = Time
- **x** = Position along the horizontal axis

This equation continuously changes over time to create moving wave effects.

2. Frame Update Algorithm

The animation works by repeatedly updating and redrawing the screen. A timer function is used to refresh frames continuously.

Steps:

1. Clear the screen.
2. Calculate new wave positions.
3. Draw updated waves.
4. Move the boat or floating object.
5. Refresh the display repeatedly.

This creates smooth real-time animation.

3. Object Translation Algorithm

Translation is used to move the boat horizontally across the water surface.

Translation formula:

$$x'= x + tx$$

$$y'= y + ty$$

Where:

- tx = Horizontal movement
- ty = Vertical movement

Implementation:

Programming Language Used:

The project is implemented using the **C++** programming language. C++ is widely used in computer graphics because it provides fast execution speed and supports graphics libraries efficiently.

Graphics Libraries and Tools Used:

1. OpenGL

OpenGL is used for rendering graphics and drawing the animated objects such as waves and boats.

2. GLUT

OpenGL Utility Toolkit is used for:

- Creating windows
- Handling display functions
- Managing animation timing
- Processing user interaction

3. Code::Blocks & Visual Studio

Code::Blocks or Microsoft Visual Studio can be used to write, compile, and run the program.

Implementation Steps:

Step 1: Initialize OpenGL Window

An OpenGL display window is created using GLUT functions. The background color and screen size are initialized.

Step 2: Draw Sinusoidal Waves

The sine function is used to generate multiple wave points across the screen.

Wave equation used:

$$y=A \sin (Bx + Ct)$$

The value of time continuously changes to animate the waves.

Step 3: Create Boat Object

A simple boat is created using:

- Triangles
- Polygons
- Lines

The boat is positioned above the moving water waves.

Step 4: Animate the Waves

A timer or idle function continuously updates:

- Wave position
- Boat movement
- Screen refresh

This creates smooth water motion.

Step 5: Render the Scene
All graphical objects such as:

- Water waves
- Boat
- Sky/background

are rendered repeatedly using OpenGL drawing functions.

Main OpenGL Functions Used:

Function	Purpose
glBegin()	Starts drawing objects
glEnd()	Ends drawing
glVertex2f()	Defines points
glColor3f()	Sets object color
glClear()	Clears screen
glFlush()	Displays graphics
glutDisplayFunc()	Handles display
glutTimerFunc()	Controls animation timing

Results / Outputs:

The Water Wave Animation project successfully produces a realistic simulation of moving water using sinusoidal wave patterns. The output shows smooth and continuous wave motion that resembles natural river or sea waves.

Key Results

1. The waves are generated using sine functions and move continuously from left to right.
2. The animation runs in real-time without lag due to continuous frame updates.
3. The water surface appears dynamic and realistic compared to static graphics.
4. A boat or floating object moves along with the waves, increasing visual realism.
5. The combination of multiple waves creates a natural river-like effect.

Output Features

- Smooth sinusoidal wave motion
- Continuous animation using OpenGL
- Moving boat synchronized with waves
- Real-time screen refresh
- Natural-looking water surface effect

Conclusion:

The Water Wave Animation project demonstrates how mathematical concepts and computer graphics techniques can be combined to create realistic natural effects. By using sinusoidal wave functions, the project successfully simulates continuous and smooth water motion similar to rivers and seas.

Through this project, we learned how animation is implemented in OpenGL using C++, including rendering, frame updates, and object movement. The addition of a floating boat improves the realism and visual appeal of the scene.

This project helps in understanding important concepts such as wave generation, real-time animation, and graphical transformations. It also shows how simple mathematical equations can produce complex and visually attractive animations in computer graphics.

Overall, the project is a successful implementation of water wave simulation and can be

Code:

```
#include <windows.h>
#include <GL/glut.h>
#include <cmath>

float t = 0.0f;

float wave(float x, float A, float B, float C, float phase)
{
    return
        A * sin(B * x + C * t + phase)
        + 0.03f * sin(12.0f * x + 1.8f * t)
        + 0.015f * sin(25.0f * x - 2.5f * t);
}

void drawSkyGradient()
{
    glBegin(GL_QUADS);

    glColor3f(0.05f, 0.25f, 0.70f);
    glVertex2f(-1.0f, 1.0f);
    glVertex2f( 1.0f, 1.0f);

    glColor3f(0.75f, 0.90f, 1.00f);
    glVertex2f( 1.0f, -0.10f);
    glVertex2f(-1.0f, -0.10f);

    glEnd();
}

void drawSea(float A, float B, float C, float phase,
             float r, float g, float b)
{
```

```
    glColor3f(r, g, b);
    glBegin(GL_TRIANGLE_STRIP);
    for(float x = -1.0f; x <= 1.0f; x += 0.01f)
    {
        float y = wave(x, A, B, C, phase);

        glVertex2f(x, -1.0f);
        glVertex2f(x, y);
    }
    glEnd();

    glLineWidth(2);
    glColor3f(1.0f, 1.0f, 1.0f);
    glBegin(GL_LINE_STRIP);

    for(float x = -1.0f; x <= 1.0f; x += 0.01f)
    {
        float y = wave(x, A, B, C, phase);
        glVertex2f(x, y);
    }

    glEnd();
}

void drawBoat()
{
    glColor3f(0.6, 0.3, 0.1);
    glBegin(GL_POLYGON);
        glVertex2f(-0.1, 0.05);
        glVertex2f( 0.1, 0.05);
        glVertex2f( 0.0, 0.15);
    glEnd();

    glColor3f(0.3, 0.2, 0.1);
    glBegin(GL_QUADS);
        glVertex2f(-0.15, 0);
        glVertex2f( 0.15, 0);
        glVertex2f( 0.10, 0.05);
        glVertex2f(-0.10, 0.05);
    glEnd();

    glBegin(GL_POLYGON);
        glVertex2f(-0.20, 0.10);
        glVertex2f(-0.10, 0.05);
        glVertex2f(-0.15, 0);
    glEnd();

    glBegin(GL_POLYGON);
        glVertex2f(0.20, 0.10);
        glVertex2f(0.10, 0.05);
```

```
        glVertex2f(0.15, 0);
    glEnd();

}

void display()
{
    glClear(GL_COLOR_BUFFER_BIT);

    drawSkyGradient();

    // Back wave
    drawSea(0.10f, 4.0f, 1.2f, 0.0f,0.00f, 0.25f, 0.70f);

    // Middle wave
    drawSea(0.07f, 6.0f, 2.0f, 2.0f, 0.00f, 0.45f, 0.85f);

    // Front wave
    drawSea(0.04f, 9.0f, 3.0f, 4.0f,0.30f, 0.70f, 1.00f);

    float boatX = 0.0f;
    float boatY = wave(boatX, 0.07f, 6.0f, 2.0f, 2.0f);

    float dx = 0.01f;

    float y1 = wave(boatX - dx, 0.07f, 6.0f, 2.0f, 2.0f);
    float y2 = wave(boatX + dx, 0.07f, 6.0f, 2.0f, 2.0f);

    float angle = atan2(y2 - y1, 2 * dx) * 180.0f / 3.1415926f;

    glPushMatrix();
        glTranslatef(boatX, boatY + 0.02f, 0.0f);
        glRotatef(angle * 0.6f, 0.0f, 0.0f, 1.0f);
        drawBoat();
    glPopMatrix();

    glFlush();
}

void update(int value)
{
    t += 0.03f;

    glutPostRedisplay();
    glutTimerFunc(16, update, 0);
}

void init()
{
    glClearColor(0.8f, 0.9f, 1.0f, 1.0f);
```

```
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-1, 1, -1, 1);
    glMatrixMode(GL_MODELVIEW);
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize(1000, 700);
    glutCreateWindow("Wave Animation");
    init();
    glutDisplayFunc(display);
    glutTimerFunc(0, update, 0);
    glutMainLoop();

    return 0;
}
```

Output: